



MamaFi

Security Assessment

CertiK Assessed on Dec 24th, 2024





Certik Assessed on Dec 24th, 2024

MamaFi

The security assessment was prepared by Certik, the leader in Web3.0 security.

Executive Summary

TYPES

Vesting

ECOSYSTEM

Ethereum (ETH)

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Delivered on 12/24/2024

KEY COMPONENTS

N/A

CODEBASE

[update](#)[base](#)[View All in Codebase Page](#)

COMMITTS

[0x9b74842E729241cE95542d097e61723a6443063c](#)[0x37c6e0b4fa8c98f177763077cd4cd381cc95884f](#)[View All in Codebase Page](#)

Vulnerability Summary



7

Total Findings

5

Resolved

2

Mitigated

0

Partially Resolved

0

Acknowledged

0

Declined

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

2 Major

2 Mitigated



Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.

1 Medium

1 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

2 Minor

2 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

2 Informational

2 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | MAMAFI

I **Summary**

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

I **Findings**

[MAM-09 : Centralization Risks in Mamafi.sol](#)

[MAM-10 : Initial Token Distribution](#)

[MAM-08 : `transferFrom\(\)` always reverts due to incorrect allowance modification](#)

[MAM-04 : Redundant Code Components](#)

[MAM-11 : `transfer\(\)` denies 0 amount transfers](#)

[MAM-06 : Optimize Contract Pausing Logic](#)

[MAM-07 : Redundant Code Components](#)

I **Formal Verification**

[Considered Functions And Scope](#)

[Verification Results](#)

I **Appendix**

I **Disclaimer**

CODEBASE | MAMAFI

Repository

update

base

Commit

0x9b74842E729241cE95542d097e61723a6443063c

0x37c6e0b4fa8c98f177763077cd4cd381cc95884f

AUDIT SCOPE | MAMAFI

1 file audited ● 1 file with Mitigated findings

ID	Repo	File	SHA256 Checksum
● MAM	mainnet	 contracts/Mamafi.sol	ec092460a1f40fb684ac02791e9e9a1f5d7d32 69224b2941ea4ed9731a2eb0e9

APPROACH & METHODS | MAMAFI

This report has been prepared for MamaFi to discover issues and vulnerabilities in the source code of the MamaFi project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | MAMAFI

7
Total Findings0
Critical2
Major1
Medium2
Minor2
Informational

This report has been prepared to discover issues and vulnerabilities for MamaFi. Through this audit, we have uncovered 7 issues ranging from different severity levels. Utilizing the techniques of Static Analysis & Manual Review to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
MAM-09	Centralization Risks In Mamafi.Sol	Centralization	Major	● Mitigated
MAM-10	Initial Token Distribution	Centralization	Major	● Mitigated
MAM-08	<code>transferFrom()</code> Always Reverts Due To Incorrect Allowance Modification	Logic Error	Medium	● Resolved
MAM-04	Redundant Code Components	Volatile Code	Minor	● Resolved
MAM-11	<code>transfer()</code> Denies 0 Amount Transfers	Coding Issue	Minor	● Resolved
MAM-06	Optimize Contract Pausing Logic	Coding Style	Informational	● Resolved
MAM-07	Redundant Code Components	Volatile Code	Informational	● Resolved

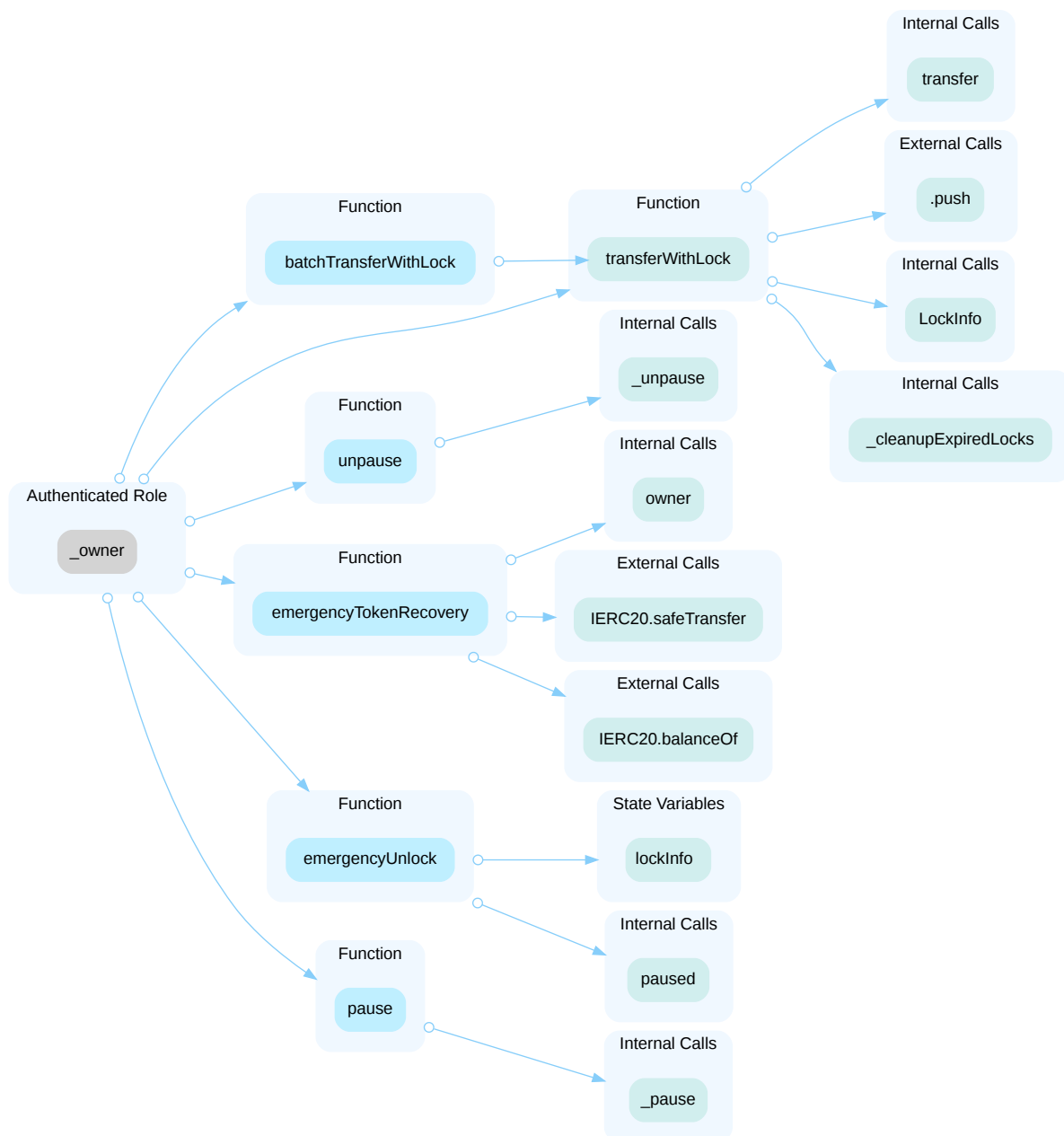
MAM-09 | CENTRALIZATION RISKS IN MAMAFI.SOL

Category	Severity	Location	Status
Centralization	● Major	contracts/Mamafi.sol (base): 70 , 78 , 97 , 196 , 200	● Mitigated

Description

In the contract `Mamafi`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and perform:

- lock tokens with specified durations
- recover tokens
- unlock accounts



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

Alleviation

[Project team, 16-Dec-2024]:

- We will utilize Gnosis Safe's `multisig` functionality post-deployment
- The contract includes `transferOwnership()` function inherited from OpenZeppelin
- After deployment, ownership will be transferred Gnosis Safe `multisig` wallet

[CertiK, 23-Dec-2024]: The contract was deployed [here](#) by [Gnosis Safe](#) with 2/3 signers:

```
1 0x4F3715588156E67dCdc73a41D6a24a267bb3063c
2 0xe4BF7ef5621C297526Cf78DA2FD47F7d08C47670
3 0x7Eb26392bA5037028CdCEe91d04415Be12A41234
```

MAM-10 | INITIAL TOKEN DISTRIBUTION

Category	Severity	Location	Status
Centralization	● Major	contracts/Mamafi.sol (base): <u>48~49</u>	● Mitigated

Description

All of the `MamaFi` tokens are sent to the contract deployer or one or more externally owned account (EOA) addresses. This creates a centralization risk because the deployer or the owners of these EOAs can distribute tokens without the community's consensus. If these addresses are compromised, a hacker could steal and sell the tokens on the market, potentially causing significant damage to the project.

Recommendation

It is recommended that the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team should make efforts to restrict access to the private keys of the deployer account or EOAs. A multi-signature ($\frac{2}{3}$, $\frac{3}{5}$) wallet can be used to prevent a single point of failure due to a private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize the project team with a third-party KYC provider to create greater accountability.

Alleviation

[CertiK, 17-Dec-2024]: The information about the initial token distribution process can be found in the [whitepaper](#).

MAM-08 | `transferFrom()` ALWAYS REVERTS DUE TO INCORRECT ALLOWANCE MODIFICATION

Category	Severity	Location	Status
Logic Error	● Medium	contracts/Mamafi.sol (base): 177	● Resolved

Description

The call `_approve(from, _msgSender(), 0)` sets the allowance to `0` before attempting the transfer, causing `super.transferFrom(from, to, amount)` to fail consistently. Since the allowance is zeroed out, the transfer cannot proceed, resulting in a revert regardless of the input parameters. This logic error prevents any successful execution of the `transferFrom()` function.

Recommendation

We recommend removing `_approve(from, _msgSender(), 0)`.

MAM-04 | REDUNDANT CODE COMPONENTS

Category	Severity	Location	Status
Volatile Code	● Minor	contracts/Mamafi.sol (base): <u>110</u>	● Resolved

Description

In the given Solidity code, the check `if (lockEndTime <= block.timestamp) revert InvalidLockEndTime();` is unnecessary. Solidity versions `^0.8.0` and above automatically handle overflow and underflow, ensuring that `lockEndTime` will not overflow when calculated as `block.timestamp + lockDuration`. The initial validation `if (lockDuration > MAX_LOCK_DURATION) revert InvalidLockDuration();` ensures `lockDuration` is within an acceptable range, making the additional overflow check for `lockEndTime` redundant.

Recommendation

We recommend removing the redundant check.

MAM-11 | `transfer()` DENIES 0 AMOUNT TRANSFERS

Category	Severity	Location	Status
Coding Issue	● Minor	contracts/Mamafi.sol (base): 154 , 168	● Resolved

Description

According to [EIP-20](#), transfers of 0 values MUST be treated as normal transfers and fire the `Transfer` event. However, both `transfer()` and `transferFrom()` will revert if the input `amount` is 0.

Recommendation

We recommend removing of `validAmount(amount)` modifier in `transfer()` and `transferFrom()`.

MAM-06 | OPTIMIZE CONTRACT PAUSING LOGIC

Category	Severity	Location	Status
Coding Style	● Informational	contracts/Mamafi.sol (base): 64	● Resolved

Description

`whenPaused` modifier can be used instead of `require(paused(), "Contract must be paused")`. This enhances readability and reduces repetitive code, ensuring consistent enforcement of the paused state across functions.

Recommendation

We recommend using `whenPaused` instead.

MAM-07 | REDUNDANT CODE COMPONENTS

Category	Severity	Location	Status
Volatile Code	● Informational	contracts/Mamafi.sol (base): <u>175~176</u>	● Resolved

Description

```
175         uint256 currentAllowance = allowance(from, _msgSender());
176         require(currentAllowance >= amount, "ERC20: insufficient allowance");
```

The check if performed by `super.transferFrom(from, to, amount)`. Rechecking the allowance in `Mamafi.transferFrom()` is redundant.

Recommendation

We recommend removing of redundant code.

FORMAL VERIFICATION | MAMAFI

Formal guarantees about the behavior of smart contracts can be obtained by reasoning about properties relating to the entire contract (e.g. contract invariants) or to specific functions of the contract. Once such properties are proven to be valid, they guarantee that the contract behaves as specified by the property. As part of this audit, we applied formal verification to prove that important functions in the smart contracts adhere to their expected behaviors.

Considered Functions And Scope

In the following, we provide a description of the properties that have been used in this audit. They are grouped according to the type of contract they apply to.

Verification of Pausable ERC-20 Compliance

We verified properties of the public interface of those token contracts that implement the pausable ERC-20 interface. This covers

- Functions `transfer` and `transferFrom` that are widely used for token transfers,
- functions `approve` and `allowance` that enable the owner of an account to delegate a certain subset of her tokens to another account (i.e. to grant an allowance), and
- the functions `balanceOf` and `totalSupply`, which are verified to correctly reflect the internal state of the contract.

The properties that were considered within the scope of this audit are as follows:

Property Name	Title
erc20-approve-revert-zero	<code>approve</code> Prevents Approvals For the Zero Address
erc20-totalsupply-correct-value	<code>totalSupply</code> Returns the Value of the Corresponding State Variable
erc20-balanceof-correct-value	<code>balanceOf</code> Returns the Correct Value
erc20-totalsupply-succeed-always	<code>totalSupply</code> Always Succeeds
erc20-transferfrom-false	If <code>transferFrom</code> Returns <code>false</code> , the Contract's State Is Unchanged
erc20-transferfrom-never-return-false	<code>transferFrom</code> Never Returns <code>false</code>
erc20-approve-correct-amount	<code>approve</code> Updates the Approval Mapping Correctly
erc20-transfer-never-return-false	<code>transfer</code> Never Returns <code>false</code>
erc20-transfer-false	If <code>transfer</code> Returns <code>false</code> , the Contract State Is Not Changed
erc20-transfer-revert-zero	<code>transfer</code> Prevents Transfers to the Zero Address

Property Name	Title	
erc20-balanceof-change-state	balanceOf	Does Not Change the Contract's State
erc20-allowance-change-state	allowance	Does Not Change the Contract's State
erc20-transfer-exceed-balance	transfer	Fails if Requested Amount Exceeds Available Balance
erc20-totalsupply-change-state	totalSupply	Does Not Change the Contract's State
erc20-transferfrom-revert-zero-argument	transferFrom	Fails for Transfers with Zero Address Arguments
erc20-transferfrom-correct-allowance	transferFrom	Updated the Allowance Correctly
erc20-transferfrom-fail-exceed-balance	transferFrom	Fails if the Requested Amount Exceeds the Available Balance
erc20-transferfrom-fail-exceed-allowance	transferFrom	Fails if the Requested Amount Exceeds the Available Allowance
erc20-transferfrom-correct-amount	transferFrom	Transfers the Correct Amount in Transfers
erc20-transferfrom-fail-recipient-overflow	transferFrom	Prevents Overflows in the Recipient's Balance
erc20-transfer-correct-amount	transfer	Transfers the Correct Amount in Transfers
erc20-transfer-recipient-overflow	transfer	Prevents Overflows in the Recipient's Balance
erc20-approve-false	If approve	Returns false , the Contract's State Is Unchanged
erc20-allowance-succeed-always	allowance	Always Succeeds
erc20-allowance-correct-value	allowance	Returns Correct Value
erc20pausable-transferfrom-revert-paused	transferFrom	Fails for a Paused Contract
erc20-approve-succeed-normal	approve	Succeeds for Valid Inputs
erc20-approve-never-return-false	approve	Never Returns false
erc20pausable-transfer-revert-paused	transfer	Fails for a Paused Contract
erc20-balanceof-succeed-always	balanceOf	Always Succeeds

Verification Results

In the remainder of this section, we list all contracts where formal verification of at least one property was not successful. There are several reasons why this could happen:

- False: The property is violated by the project.
- Inconclusive: The proof engine cannot prove or disprove the property due to timeouts or exceptions.
- Inapplicable: The property does not apply to the project.

Detailed Results For Contract Mamafi (contracts/Mamafi.sol) In Commit 0x37c6e0b4fa8c98f177763077cd4cd381cc95884f

Verification of Pausable ERC-20 Compliance

Detailed Results for Function `approve`

Property Name	Final Result	Remarks
erc20-approve-revert-zero	● True	
erc20-approve-correct-amount	● True	
erc20-approve-false	● True	
erc20-approve-succeed-normal	● True	
erc20-approve-never-return-false	● True	

Detailed Results for Function `totalSupply`

Property Name	Final Result	Remarks
erc20-totalsupply-correct-value	● True	
erc20-totalsupply-succeed-always	● True	
erc20-totalsupply-change-state	● True	

Detailed Results for Function `balanceOf`

Property Name	Final Result	Remarks
erc20-balanceof-correct-value	● True	
erc20-balanceof-change-state	● True	
erc20-balanceof-succeed-always	● True	

Detailed Results for Function `transferFrom`

Property Name	Final Result	Remarks
erc20-transferfrom-false	● True	
erc20-transferfrom-never-return-false	● True	
erc20-transferfrom-revert-zero-argument	● True	
erc20-transferfrom-correct-allowance	● True	
erc20-transferfrom-fail-exceed-balance	● True	
erc20-transferfrom-fail-exceed-allowance	● True	
erc20-transferfrom-correct-amount	● True	
erc20-transferfrom-fail-recipient-overflow	● True	
erc20pausable-transferfrom-revert-paused	● True	

Detailed Results for Function `transfer`

Property Name	Final Result	Remarks
erc20-transfer-never-return-false	● True	
erc20-transfer-false	● True	
erc20-transfer-revert-zero	● True	
erc20-transfer-exceed-balance	● True	
erc20-transfer-correct-amount	● True	
erc20-transfer-recipient-overflow	● Inconclusive	
erc20pausable-transfer-revert-paused	● True	

Detailed Results for Function `allowance`

Property Name	Final Result	Remarks
erc20-allowance-change-state	● True	
erc20-allowance-succeed-always	● True	
erc20-allowance-correct-value	● True	

APPENDIX | MAMAFI

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

Details on Formal Verification

Some Solidity smart contracts from this project have been formally verified. Each such contract was compiled into a mathematical model that reflects all its possible behaviors with respect to the property. The model takes into account the semantics of the Solidity instructions found in the contract. All verification results that we report are based on that model.

The following assumptions and simplifications apply to our model:

- Certain low-level calls and inline assembly are not supported and may lead to a contract not being formally verified.
- We model the semantics of the Solidity source code and not the semantics of the EVM bytecode in a compiled contract.

Formalism for property specifications

All properties are expressed in a behavioral interface specification language that CertiK has developed for Solidity, which allows us to specify the behavior of each function in terms of the contract state and its parameters and return values, as well as contract properties that are maintained by every observable state transition. Observable state transitions occur when the contract's external interface is invoked and the invocation does not revert, and when the contract's Ether balance is changed

by the EVM due to another contract's "self-destruct" invocation. The specification language has the usual Boolean connectives, as well as the operator `\old` (used to denote the state of a variable before a state transition), and several types of specification clause:

Apart from the Boolean connectives and the modal operators "always" (written `[]`) and "eventually" (written `<>`), we use the following predicates to reason about the validity of atomic propositions. They are evaluated on the contract's state whenever a discrete time step occurs:

- `requires [cond]` - the condition `cond`, which refers to a function's parameters, return values, and contract state variables, must hold when a function is invoked in order for it to exhibit a specified behavior.
- `ensures [cond]` - the condition `cond`, which refers to a function's parameters, return values, and both `\old` and current contract state variables, is guaranteed to hold when a function returns if the corresponding requires condition held when it was invoked.
- `invariant [cond]` - the condition `cond`, which refers only to contract state variables, is guaranteed to hold at every observable contract state.
- `constraint [cond]` - the condition `cond`, which refers to both `\old` and current contract state variables, is guaranteed to hold at every observable contract state except for the initial state after construction (because there is no previous state); constraints are used to restrict how contract state can change over time.

Description of the Analyzed ERC-20-Pausable Properties

Properties related to function `approve`

erc20-approve-correct-amount

All non-reverting calls of the form `approve(spender, amount)` that return `true` must correctly update the allowance mapping according to the address `msg.sender` and the values of `spender` and `amount`.

Specification:

```
requires spender != address(0);
ensures \result ==> allowance(msg.sender, \old(spender)) == \old(amount);
```

erc20-approve-false

If function `approve` returns `false` to signal a failure, it must undo all state changes that it incurred before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-approve-never-return-false

The function `approve` must never returns `false`.

Specification:

```
ensures \result;
```

erc20-approve-revert-zero

All calls of the form `approve(spender, amount)` must fail if the address in `spender` is the zero address.

Specification:

```
ensures \old(spender) == address(0) ==> !\result;
```

erc20-approve-succeed-normal

All calls of the form `approve(spender, amount)` must succeed, if

- the address in `spender` is not the zero address and
- the execution does not run out of gas.

Specification:

```
requires spender != address(0);  
ensures \result;  
reverts_only_when false;
```

Properties related to function `totalSupply`

erc20-totalsupply-change-state

The `totalSupply` function in contract Mamafi must not change any state variables.

Specification:

```
assignable \nothing;
```

erc20-totalsupply-correct-value

The `totalSupply` function must return the value that is held in the corresponding state variable of contract Mamafi.

Specification:

```
ensures \result == totalSupply();
```

erc20-totalsupply-succeed-always

The function `totalSupply` must always succeeds, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `balanceOf`

erc20-balanceof-change-state

Function `balanceOf` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc20-balanceof-correct-value

Invocations of `balanceOf(owner)` must return the value that is held in the contract's balance mapping for address `owner`.

Specification:

```
ensures \result == balanceOf(\old(account));
```

erc20-balanceof-succeed-always

Function `balanceOf` must always succeed if it does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `transferFrom`

erc20-transferfrom-correct-allowance

All non-reverting invocations of `transferFrom(from, dest, amount)` that return `true` must decrease the allowance for address `msg.sender` over address `from` by the value in `amount`.

Specification:

```
ensures \result ==> allowance(\old(sender), msg.sender) == \old(allowance(sender,
msg.sender)) - \old(amount)
|| (allowance(\old(sender), msg.sender) == \old(allowance(sender,
msg.sender)) && \old(allowance(sender, msg.sender)) == type(uint256).max);
```

erc20-transferfrom-correct-amount

All invocations of `transferFrom(from, dest, amount)` that succeed and that return `true` subtract the value in `amount` from the balance of address `from` and add the same value to the balance of address `dest`.

Specification:

```
requires recipient != sender;
requires balanceOf(recipient) + amount <= type(uint256).max;
ensures \result ==> balanceOf(\old(recipient)) == \old(balanceOf(recipient) +
    amount)
    && balanceOf(\old(sender)) == \old(balanceOf(sender) - amount);
also
requires recipient == sender;
ensures \result ==> balanceOf(\old(recipient)) == \old(balanceOf(recipient));
```

erc20-transferfrom-fail-exceed-allowance

Any call of the form `transferFrom(from, dest, amount)` with a value for `amount` that exceeds the allowance of address `msg.sender` must fail.

Specification:

```
requires msg.sender != sender;
requires amount > allowance(sender, msg.sender);
ensures !\result;
```

erc20-transferfrom-fail-exceed-balance

Any call of the form `transferFrom(from, dest, amount)` with a value for `amount` that exceeds the balance of address `from` must fail.

Specification:

```
requires amount > balanceOf(sender);
ensures !\result;
```

erc20-transferfrom-fail-recipient-overflow

Any call of `transferFrom(from, dest, amount)` with a value in `amount` whose transfer would cause an overflow of the balance of address `dest` must fail.

Specification:

```
requires recipient != sender;
requires balanceOf(recipient) + amount > type(uint256).max;
ensures !\result;
```

erc20-transferfrom-false

If `transferFrom` returns `false` to signal a failure, it must undo all incurred state changes before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-transferfrom-never-return-false

The `transferFrom` function must never return `false`.

Specification:

```
ensures \result;
```

erc20-transferfrom-revert-zero-argument

All calls of the form `transferFrom(from, dest, amount)` must fail for transfers from or to the zero address.

Specification:

```
ensures \old(sender) == address(0) ==> !\result;  
also  
ensures \old(recipient) == address(0) ==> !\result;
```

erc20pausable-transferfrom-revert-paused

Any call of the form `transferFrom(from, dest, amount)` must fail for a paused contract.

Specification:

```
reverts_when paused();
```

Properties related to function `transfer`**erc20-transfer-correct-amount**

All non-reverting invocations of `transfer(recipient, amount)` that return `true` must subtract the value in `amount` from the balance of `msg.sender` and add the same value to the balance of the `recipient` address.

Specification:

```
requires recipient != msg.sender;  
requires balanceOf(recipient) + amount <= type(uint256).max;  
ensures \result ==> balanceOf(recipient) == \old(balanceOf(recipient) + amount)  
&& balanceOf(msg.sender) == \old(balanceOf(msg.sender) - amount);  
  also  
requires recipient == msg.sender;  
ensures \result ==> balanceOf(msg.sender) == \old(balanceOf(msg.sender));
```

erc20-transfer-exceed-balance

Any transfer of an amount of tokens that exceeds the balance of `msg.sender` must fail.

Specification:

```
requires amount > balanceOf(msg.sender);  
ensures !\result;
```

erc20-transfer-false

If the `transfer` function in contract `Mamafi` fails by returning `false`, it must undo all state changes it incurred before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-transfer-never-return-false

The transfer function must never return `false` to signal a failure.

Specification:

```
ensures \result;
```

erc20-transfer-recipient-overflow

Any invocation of `transfer(recipient, amount)` must fail if it causes the balance of the `recipient` address to overflow.

Specification:

```
requires recipient != msg.sender;  
requires balanceOf(recipient) + amount > type(uint256).max;  
ensures !\result;
```

erc20-transfer-revert-zero

Any call of the form `transfer(recipient, amount)` must fail if the recipient address is the zero address.

Specification:

```
ensures \old(recipient) == address(0) ==> !\result;
```

erc20pausable-transfer-revert-paused

Any invocation of `transfer(recipient, amount)` must fail if the contract is paused.

Specification:

```
reverts_when paused();
```

Properties related to function `allowance`

erc20-allowance-change-state

Function `allowance` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc20-allowance-correct-value

Invocations of `allowance(owner, spender)` must return the allowance that address `spender` has over tokens held by address `owner`.

Specification:

```
ensures \result == allowance(\old(owner), \old(spender));
```

erc20-allowance-succeed-always

Function `allowance` must always succeed, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Entire **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

